

JAIMIN DOSHI · PROFESSIONAL
PROJECT GUIDE

ALU+ FIFO

in **Verilog HDL**

An 8-bit Arithmetic Logic Unit integrated with an 8-bit
First-In First-Out buffer.

TOOLS
Quartus II · ModelSim-Altera



COVER
01/14

WHAT THIS GUIDE COVERS

Contents

Ten sections walking through the 8-bit ALU + FIFO project — from motivation and architecture, through to simulation results.

01 Project Overview and Motivation

02 Objectives and Learning Outcomes

03 Project Architecture

04 ALU — Detailed Working

05 FIFO — Detailed Working

06 Top-Level Integration

07 Testbench and Verification

08 Quartus and ModelSim Workflow

09 Explanation of Simulation Results

10 Conclusion

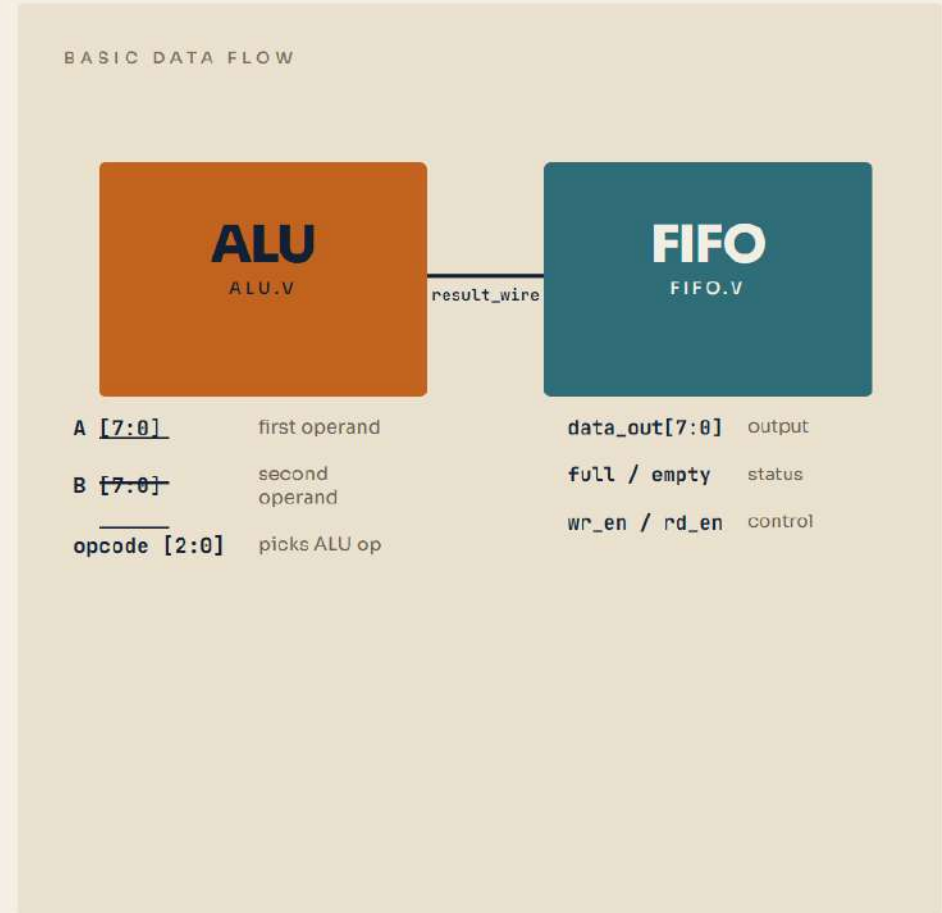
10

SECTIONS

A small ALU feeding a small queue.

The project implements an 8-bit ALU and an 8-bit FIFO using Verilog HDL. The ALU receives two 8-bit operands and a 3-bit opcode, performs the selected operation, and produces an 8-bit result. That result is connected to the FIFO input.

The FIFO stores results and returns them in the same order in which they were written — combining **combinational logic** with **sequential, state-based logic**.



What this project is meant to teach.

OUTCOMES COUNTED

10

distinct goals across design, verification, and tooling.

FOUNDATIONS

Understand combinational and sequential digital circuits.

Design an 8-bit ALU using Verilog HDL.

Use an opcode to select different operations.

STORAGE & WIRING

Design and use a FIFO for temporary storage.

Connect independent modules through a top-level module.

VERIFICATION

Create a clocked Verilog testbench and apply test vectors.

Use Quartus for analysis and synthesis.

Use ModelSim to inspect waveforms and transcript output.

DEBUG & EXTEND

Debug common undefined-module, top-level and initialization errors.

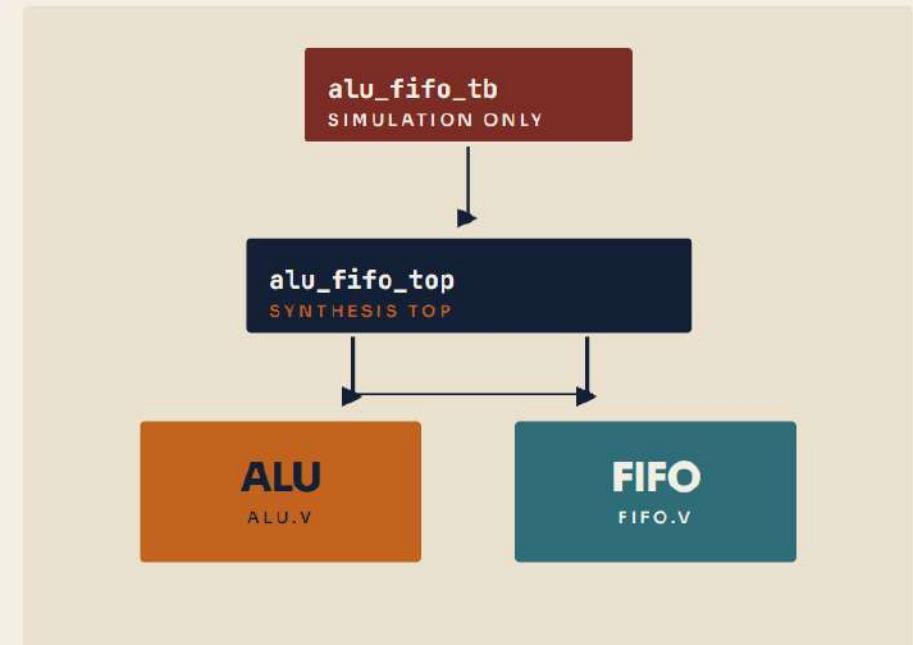
Extend the project toward an FPGA / VLSI-oriented workflow.

Three hardware modules, one simulation testbench.

FILES & MODULES

File	Module	Purpose
alu.v	alu	Performs arithmetic / logical operations.
fifo.v	fifo	Stores and reads results in order.
alu_fifo_top.v	alu_fifo_top	Connects ALU and FIFO together.
alu_fifo_tb.v	alu_fifo_tb	Drives simulation and checks behaviour.

HIERARCHY



Six signals in, out, and status.

An Arithmetic Logic Unit is the computation block. Two 8-bit operands and a 3-bit opcode come in; an 8-bit result, a carry, and a zero flag come out.

ALU

Signal	Width	Direction	Purpose
A	8	Input	First operand
B	8	Input	Second operand
opcode	3	Input	Selects the operation
result	8	Output	Operation result
carry	1	Output	Carry / status
zero	1	Output	Indicates a zero result

Three opcode bits, four operations verified.

OPCODE WIDTH

3 bit

Up to eight operation codes—
four verified, others reserved.

Opcode	Operation	Example	Expected
000	ADD	0A + 05	0F
001	SUB	14 - 08	0C
010	AND	AA & CC	88
011	OR	AA CC	EE

The exact opcode mapping used in a presentation should always match the actual **alu.v** file.
Codes **100–111** remain available for future ADD-with-carry, XOR, shifts, etc.

A case-statement picks it; flags describe the result.

ALU.V — BODY
FRAGMENT

```
case (opcode)
  3'b000: result = A + B;
  3'b001: result = A - B;
  3'b010: result = A & B;
  3'b011: result = A | B;
  default: result = 8'b0;
```

endcase

The default arm maps any unused opcode to zero — safe fallback.

FLAGS & STATUS

zero

Asserted when the result equals zero. Useful for branch logic later.

carry

Primarily associated with unsigned arithmetic — ADD, SUB-with-borrow.

overflow

Signed overflow is a separate condition — can be added as an improvement. A stronger ALU exposes N, Z, C, V flags.

The carry and zero outputs are exposed today. Negative/sign and overflow remain extension points.

First written, first read out.

A FIFO works like a queue: the **first value written** is the **first value read**.

The ALU produces results faster than the next block may consume them — the FIFO acts as a buffer so several results can wait safely until they are read.

MAIN SIGNALS

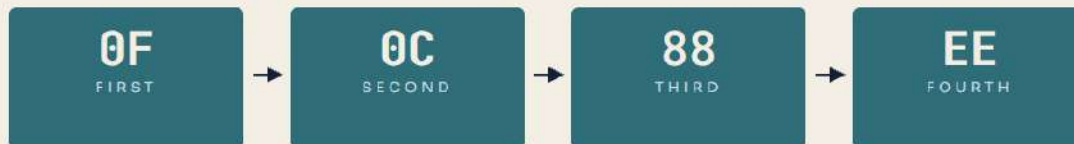
clk sync state	reset initialise	data_in data being stored	data_out data returned
write_en request a write	read_en request a read	full capacity reached	empty no valid data

FIFO depth describes how many results can wait at once. **full** and **empty** keep the writer from overflowing and the reader from reading past real data.

Order in equals order out.

The values 0F → 0C → 88 → EE are written in sequence; reads return them in the same sequence — the FIFO never reorders.

WRITE



READ



Therefore the FIFO ordering is correct. Read-out exactly matches write-in — a property you can read across the waveform window.

One wire joins two modules.

The top-level module instantiates the ALU and the FIFO. The internal wire `result_wire` carries the ALU result into the FIFO and also drives the top-level `alu_result` output.

ALU_FIFO_TOP.V

```

wire [7:0] result_wire;
assign alu_result = result_wire;
alu alu ( .A(A), .B(B), .opcode(opcode),
          .result(result_wire), .carry(carry), .zero(zero) );
fifo fifo ( .clk(clk), .reset(reset),
            .data_in(result_wire), .write_en(write_en), .read_en(read_en),
            .data_out(data_out), .full(full), .empty(empty) );

```

READING THE FILE

Two named instances — `alu` and `fifo` — share one wire.

The same wire also drives `alu_result` on the top-level output.

Named port connections like `.result(result_wire)` keep the wiring explicit and reduce mistakes.

Drive the design, watch the waveforms.

§ 07.1 — DUT INSTANTIATION

```
alu_fifo_top DUT (  
    .clk(clk), .reset(reset),  
    .A(A), .B(B), .opcode(opcode),  
    .write_en(write_en), .read_en(read_en),  
    .alu_result(alu_result), .data_out(data_out),  
    .carry(carry), .zero(zero),  
    .full(full), .empty(empty)  
);
```

Testbench instantiation connects testbench signals to the top-level ALU + FIFO design under test.

§ 08.1 — REQUIRED SOURCE FILES

alu.v

Arithmetic Logic Unit

fifo.v

First-In First-Out buffer

alu_fifo_top.v

Top-level wiring

alu_fifo_tb.v

Simulation testbench — not synthesised as hardware

What the waveforms showed.

§ 09.1 — ALU TEST RESULTS

Test	A	B	Opcode	ALU result
ADD	0A	05	000	0F
SUB	14	08	001	0C
AND	AA	CC	010	88
OR	AA	CC	011	EE

§ 09.2 — FIFO ORDERING

WRITE: 0F → 0C → 88 → EE

READ : 0F → 0C → 88 → EE

FIFO ordering is correct.

§ 09.3 — FLAGS

empty

starts 1 → drops to 0
on first write
→ returns to 1 drained

full

stayed 0 — capacity
not reached in this run

§ 09.4 — TIMING NOTE

The ALU result updates **combinatorially** when A, B or opcode changes.

The FIFO stores values only on its **clock edges**.

What we proved.

And where it can go.

VERIFIED EXPRESSIONS

0A + 05 = 0F

ADD verified

14 - 08 = 0C

SUB verified

AA & CC = 88

AND verified

AA | CC = EE

OR verified

FIFO VERIFIED

WRITE 0F → 0C → 88 → EE · READ 0F → 0C → 88 → EE

FOUNDATION FOR

additional ALU functions, stronger FIFO handling, improved verification, and FPGA implementation.

PROJECT SUMMARY

8-bit ALU + FIFO

Verilog HDL

Quartus II

ModelSim

RTL Design

Functional Verification